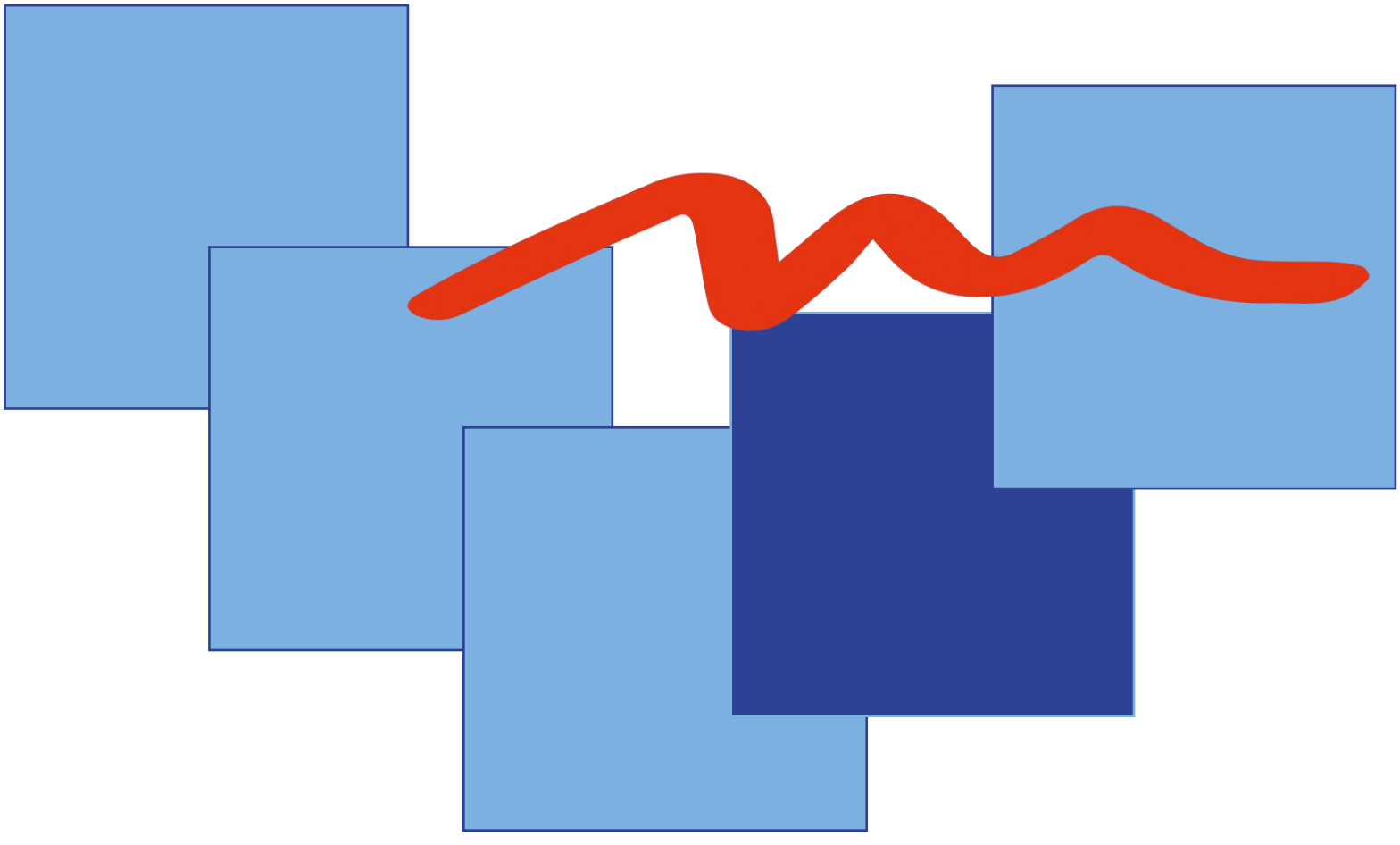
Anwendungsentwicklung und Programmierung

Jahrgangsstufe 10

Schuljahr 2026 / 2027

1. Einstieg in die Programmierung



1.1 Theoretische Grundlagen von Programmiersprachen

Bevor Sie sich konkret mit einer Programmiersprache beschäftigen, sollen Sie sich erstmal über verschiedene Eigenschaften von Programmiersprachen informieren.

Aufgaben:

1. Bearbeiten Sie in Ihrer Gruppe die zugewiesene Aufgabe. Im Anschluss werden Sie mit Mitgliedern der anderen Gruppen gemischt und sollen dort Ihre Ergebnisse den anderen erklären.

   Gruppe A: Arten von Programmiersprachen
   Erklären Sie kurz den Ansatz folgender Programmiersprachen und nennen Sie für jede Sprache konkrete Beispiele.

   - Strukturierte Programmiersprachen

     ```
     Programme werden in baumartige Strukturen zerlegt. Paradigma der
     prozeduralen Programmierung enthalten. Strukturen dürfen lediglich
     aus 3 Strukturen bestehen: Sequenz, Verzweigung, Schleifen
     ```

   - Imperative Programmiersprachen

     ```
     Folge aus Anweisungen, wann was getan werden soll.
     Kontrollstrukturen für Befehlsausführung.


     Bsp.: BASIC (C64)
     ```

   - Deklarative Programmiersprachen

     ```
     Beschreibung des Problems im Vordergrund. Lösung wird automatisch
     ermittelt. Berechnung steht im Gegensatz zu imperativen Sprachen im
     Vordergrund, nicht das Wie.

     Bsp.: SQL (nicht sagen wie Daten aus Datenbanken geholt werden,
     ```

   - Objektorientierte Programmiersprachen

     ```
     Abbildung der Wirklichkeit. Objekte der Wirklichkeit in Klassen
     definiert. Objekte bestehen aus Attributen & Funktionalitäten.
     Mächtiges Konzept durch Polymorphie & Vererbung.
     Bsp.: Java
     ```

   - Funktionale Programmiersprachen

     ```
     Mathematischer Begriff der Funktion im Vordergrund. Funktion kann
     definiert, angewendet, verknüpft & ___ werden.
     ```

Gruppe B: Implementierungsformen von Software
Erklären Sie kurz (incl. Vor- bzw. Nachteilen) die folgenden Implementierungsformen von Software und nennen Sie für jede Implementierungsform typische Programmiersprachen als Beispiel.

- Compiler

```
PC-Programm das Quellcode in Form übersetzt, die vom PC gelesen
werden kann. Gewährleistet, dass Quellcode syntaktisch richtig ist.
Compiler kann Codeoptimierung automatisch vornehmen.
Vorteil: Speicherplatzoptimierung, hohe Ausführungsgeschwindigkeit
Nachteil: Kompilierzeit, Abhängigkeit von der Zielplattform
```

- Interpreter

```
Versucht Programmzeilen nacheinander auszuführen. Hierfür
Übersetzung in Maschinencode. Abbruch bei Fehlern.
Vorteil: einfaches Debugging
Nachteil: langsamere Programmausführung
```

- JIT-Compiler

```
Teilprogramme zur Laufzeit in Maschinencode übersetzt. Ziel ist es
Ausführungsgeschwindigkeit zu optimieren. (Kompilierung von Teilen,
nicht allem). Meist in Kombi mit VM.
Vorteil: Software wird durch VM plattformunabhängiger
Nachteil: langsamer als native Compiler
```

Gruppe C: Typisierung von Variablen
Erklären Sie kurz die unterschiedliche Arten von Variablen-Typisierungen und nennen Sie für jede Art eine passende Programmiersprache.

- starke vs schwache Typisierung

```
wie streng werden Typen unterschieden. Welche Datentypen können
ineinander umgewandelt werden? Werden Datentypen implizit
ineinander umgewandelt?
stark: Java, C#
schwach: C, Pearl
```

- dynamische vs statische Typisierung

```
bei statischer Typisierung werden Datentypen bei der Übersetzung
geprüft, bei dynamischer zu Laufzeit. Dies wirkt sich auf das
Finden von Zuweisungsfehlern aus.
dynamisch: Python, PHP
statisch: Java, C
```

- explizite vs implizite Typisierung

```
explizit muss Datentypen genau festlegen. Bei implizit werden sie
anhand der Zuweisung abgeleitet.
explizit: Pascal, Java
implizit: PHP, Python
```

1.2 Das erste Java-Programm

Während Ihrer Berufsschulzeit lernen Sie einige Programmierkonzepte kennen, welche mit der Programmiersprache Java verdeutlicht werden sollen. Dafür sollen Sie nun ein erstes ausführbares Java-Programm erstellen. Ein ausführbares Java-Programm ist eine Klasse mit einer so genannten `main`-Methode. Um eine solche Klasse anzulegen, muss erstmal ein neues Java-Projekt in Eclipse angelegt werden. In dieser `main`-Methode steht der Programmcode, den das Java-Programm ausführen soll. Soll beispielsweise der Text „Hello World“ auf der Konsole ausgegeben werden, verwendet man beispielsweise den Befehl `System.out.println("Hello World")`:

```
public class MeinErstesProgramm{
  public static void main(String[] args){
    //Im main-Block steht der Programmcode, der ausgeführt werden soll
    System.out.println("Hello World");
    //Weitere Anweisungen...
  }
}
```

Erstellen Sie ein neues Projekt `Einstieg` und legen Sie in diesem Projekt eine Klasse `MeinErstesProgramm` mit einer `main`-Methode an, welche „Hello World“ auf der Konsole ausgibt.

Hello.java -> Main Java Datei
Hello.class -> by-code??

1.3 Genauere Betrachtung der Konsolenausgabe

Im folgenden Abschnitt soll die Konsolenausgabe genauer betrachtet werden.

Aufgaben:

1. Erstellen Sie eine Klasse mit dem Namen `ErweiterteKonsolenausgabe`, die eine `main`-Methode enthält.

2. Führen Sie folgende Zeilen Code in der `main`-Methode aus und beschreiben Sie den Unterschied zwischen den Methoden `System.out.print` und `System.out.println`:

   ```
   System.out.println("Hello");
   System.out.println("World");
   System.out.print("Hello");
   System.out.print("World");
   ```

   ```
   System.out.println = prints every content in a line
   System.out.print = prints all content together in the same line
   ```

3. Führen Sie folgende Zeilen Code in der `main`-Methode aus und beschreiben Sie die Auswirkung von `\n` und `\t` bei folgenden Ausgaben:

```
System.out.print("Hello\n");
System.out.println("World");
System.out.print("Hello\t");
System.out.println("World");
```

\n = Escape-Sequenz

```
\n = line break even with System.out.print

\t = tab between content and next System.out.println
```

4. Welche Ausgabe liefern jeweils die dargestellten Programmzeilen:

```
System.out.println("1");
System.out.print("2");
System.out.print("3");
System.out.println("4");
```

```
1

rest concatenated to 234
```

```
System.out.print("1\n");
System.out.print("2");
System.out.print("3\n");
System.out.print("4\t");
```

```
1
23
4
```

```
System.out.println("1\n");
System.out.print("2\n");
System.out.print("3\n");
System.out.println("4\n");
```

```
1

2
3
4
```

```
System.out.println("1");
System.out.println("2\n");
System.out.print("3");
System.out.println("4");
```

```
1
2

34
```

5. Erzeugen Sie mithilfe von `System.out.print`- bzw. `System.out.println`-Methoden folgende Konsolenausgabe:

```
      \_ _/ \_ _/
      (" ") (" ")
 .-----\ /   \ /-----.
/|______|     |______|\
" | \ ||       || / |  "
```

```
System.out.println("\t  \\_ _/  \\_ _/");
System.out.println("\t  (\" \")  (\" \")");
System.out.println(" .-----\\ /\t\\ /-----.");
System.out.println("/|_____|\t|_____|\\ ");
System.out.println("\" | \\ ||\t\t|| / |  \"");
```

1.4 Variablen und Datentypen

In der Programmierung benötigt man sehr oft **Variablen** für Berechnungen, Abläufe,... etc. Jede Variable hat dabei einen Datentyp. Im Folgenden finden Sie einen Überblick über die gängigsten elementaren primitiven Datentypen in Java.

primitive Datentypen

Links steht Datentyp, rechts Literal (int, double)

Typ	Bit-Tiefe	Wertebereich
numerisch		
Ganzzahlen		
int	32 Bit	-2^{31} bis $2^{31}-1$
long	64 Bit	-2^{63} bis $2^{63}-1$
byte	8 Bit	-2^{7} bis $2^{7}-1$
(short)	16 Bit	-2^{15} bis $2^{15}-1$
Fließkommazahlen		
double	64 Bit	$\approx +/- 1,78 \cdot 10^{308}$
(float)	32 Bit	$\approx +/- 3,4 \cdot 10^{38}$
logisch		
boolean	JVM-spezifisch	true, false
Text		
char	16 Bit	Einzelnes Unicode-Zeichen

ASCII => 127 Zeichen
UTF-8 => Standard -> Zeichenlängen \= Stringlänge
UTF-16 => UTF-8 + Emojis

-> -1 um die 0 einzubeziehen

Bereich von -128 bis 127. z.B. Alter

am häufigsten verwendet -> bei Fließkomma

Für Tests: wissen, wie viele Daten ungefähr in verschiedene Wertebereiche reinpassen

Neben dem Datentyp benötigt eine Variable einen eindeutigen Namen (keine Systemwörter). Möchte man beispielsweise eine Variable mit dem Namen `alter` vom Datentyp `int`, eine Variable `geschlecht` vom Datentyp `char` oder eine Variable `durchschnitt` vom Datentyp `double` in Java **deklarieren**, geschieht das folgendermaßen:

```
int alter;
char geschlecht;
double durchschnitt;
```

Möchten man diese Variablen nach der Deklaration **initialisieren** bzw. einen Wert zuweisen, so wird dafür in Java der **= Operator** verwendet:

```
int alter;
char geschlecht;
alter = 29;
geschlecht = 'w';
double durchschnitt;
durchschnitt = 2.5;
```

bei char => ein Zeichen wird von single quotes eingeschlossen, mehr von double quotes

Häufig geschieht die Deklaration und Initialisierung zusammen in einer Zeile:

```
int alter = 29;
char geschlecht = 'w';
double durchschnitt = 2.5;
```

Soll der Inhalt einer Variable auf der Konsole ausgegeben werden, so kann auch hierfür die `System.out.println` bzw. `System.out.print` Methode verwendet werden:

```
int alter = 29;
System.out.println(alter);
```

Für Zeichenketten bzw. Texte benötigt man den nicht primitiven Datentyp **String**. Strings werden in Anführungszeichen geschrieben:

```
String name = "Robert";
```

Aufgaben:

1. Erklären Sie kurz, warum folgende Wertzuweisung von der Variable b nicht funktioniert.

   ```
   int x = 24;
   byte b = x;
   ```

   ```
   x hat den Datentyp integer und kann nicht byte zugeordnet werden, da
   int einen zu großen Wertebereich für byte hat.

   so geht's:
   byte = (byte)x
   ```

2. Gegeben ist folgender zusammenhängender Codeausschnitt mit Zeilennummern in einer `main`-Methode. Einige Zeilen sind jedoch fehlerhaft. Korrigieren Sie diese.

   Hinweis

   Sie müssen insgesamt 9 Zeilen korrigieren

   ```
   1  int a = 24.5;
   2  boolean boo = 20;
   3  int g = 17;
   4  int y = g;
   5  g = g + g;
   6  byte b = 3;
   7  byte v = 'b';
   8  int a = 10;
   9  char u = '\u4567';
   10 double f = 32,5;
   11 char c = "t";
   12 int m = 2_147_483_647;
   13 long l = 2_147_483_648;
   14 float fl = 32.5;
   15 boolean c = true;
   16 String super = "1";
   17 char ch = 65;
   ```

   ```
   1. Zeile 1: Kommazahlen werden mit double deklariert
   2. Zeile 2: boolean sollte true/false stehen
   3. Zeile 4: kann weggelassen werden oder Zeile 5 y = y + y
   4. Zeile 8: a wird redeklariert, was unmöglich ist
   5. Zeile 10: Komma statt Punkt verwendet
   6. Zeile 11: char werden mit '' deklariert, strings mit ""
   7. Zeile 13: L oder l fehlt, da Zahl sonst als int interpretiert wird
   8. Zeile 14: F oder f am Ende fehlt
   9. Zeile 15: c wurde schon deklariert
   10. Zeile 16: super ist ein Keyword zur Deklaration einer parent-method
   ```

3. Was wird bei folgendem Programm auf der Konsole ausgegeben? Erklären Sie die unterschiedliche Bedeutung des + Operators.

```
int i = 5;
int j = 3;
String s = "3";
System.out.println(i + j);
System.out.println(s + i);
System.out.println(i + j + s);
System.out.println(i + s + j);
```

```
8
35
83
533
+ zum Konkatenieren zweier Literale?/ Werte (egal ob string oder int).
```

4. Welchen Wert besitzen die Variablen `i` und `j` nach folgenden Zeilen Java-Code? Erklären Sie diese Werte.

```
int i = 2_147_483_647;
i = i + 1;
int j = -2_147_483_648;
j = j - 1;
```

```
i => Overflow Error, da Ende Wertebereich erreicht ist -> springt ins
Negative
i = -2_147_483_647
j = +2_147_483_648
```

5. **Für Experten:**
Ergänzen Sie für die Variablen passende Datentypen und erklären Sie anschließend die Konsolenausgaben.

```
int  i = 036;
System.out.println(i);
     j = 0x1c;
System.out.println(j);
     d = .1E-1;
System.out.println(d);
     x = 10^3;
System.out.println(x);
```

```
     k = 5 | 3;
System.out.println(k);
     m = 5 & 3;
System.out.println(m);
     n = 3 << 2;
System.out.println(n);
```

1.5 Programmierübung: Notendurchschnitt berechnen

1.5.1 Einstieg

Im Folgenden soll ein Java-Programm entwickelt werden, welches anhand der Noten in einem Fach den entsprechenden Schnitt und damit verbundene Zeugnisnote ermittelt. Dafür wurde schon folgendes Codegerüst erstellt:

```
1  public class Notenberechnung{
2    public static void main (String[] args){
3    //Noteninformationen wurden wie folgt eingelesen
4    int noteTest1 = 3, noteTest2 = 2;
5    int noteSA1 = 4, noteSA2 = 2;
6    int anzahl = 6;
7
8    int summeTest = noteTest1 + noteTest2;
9    int summeSA = noteSA1 + noteSA2;
10
11   double durchschnitt = (summeTest + summeSA*2)/anzahl;
12   System.out.println("Ihr Notendurchschnitt beträgt: " + durchschnitt);
13   }
14 }
```

Aufgaben:

1. Obiges Programm liefert keinen korrekten Notendurchschnitt. Erklären Sie kurz warum und korrigieren Sie das Programm. Die Zeilen 4 bis 9 dürfen Sie dabei **nicht** verändern.

2. Die Ausgabe des Notendurchschnitts mit 16 Nachkommastellen ist nicht effektiv. Daher soll der Notendurchschnitt auf zwei Stellen nach dem Komma richtig gerundet werden. Passen Sie Ihr Programm dahingehend an. Der gerundete Durchschnitt soll dabei auch in einer Variable gespeichert werden.

3. Ergänzen Sie Ihr Programm um die Ausgabe der Zeugnisnote, welche sich aus dem Notendurchschnitt ergibt. Beachten Sie dabei, dass bei Komma fünf die bessere Note gegeben werden soll.
   Dies bedeutet, dass beispielsweise ein Notendurchschnitt von 2,50 die Zeugnisnote 2 und ein Zeugnisnotendurchschnitt von 2,51 die Zeugnisnote 3 ergeben soll.

1.5.2 Vertiefung

Das Programm soll nun erweitert werden, so dass der Benutzer beliebig viele Noten eingeben kann. Anschließend sollen alle eingegebenen Noten mit dem daraus resultieren gerundeten Notendurchschnitt und der entsprechenden Zeugnisnote ausgegeben werden.
Aufgaben:

1. Für Benutzereingaben kann beispielsweise die Scanner-Klasse verwendet werden. Folgendes Programm zeigt einen Einsatz der Scanner-Klasse:

```
import java.util.Scanner; // Ganz oben im Java-File, VOR der Klasse

public class Benutzereingaben {
  public static void main (String[] args){
    Scanner scanner = new Scanner(System.in); // Erstellt ein Scanner-Objekt

    System.out.println("Bitte geben Sie Ihren Namen ein:");
    String name = scanner.nextLine(); // Liest eine ganze Zeile Text
    System.out.println("Hallo, " + name + "!");

    System.out.println("Bitte geben Sie Ihr Alter ein:");
    int alter = scanner.nextInt(); // Liest eine ganze Zahl
    System.out.println("Sie sind " + alter + " Jahre alt.");

    scanner.close(); // Wichtig: Scanner am Ende schließen!
  }
}
```

   (a) Erklären Sie kurz wie die Benutzereingabe mithilfe des Scanner funktioniert.

   (b) Passen Sie das obige Programm so an, dass zuerst das Alter und anschließend der Name eingegeben wird. Warum kommt es hierbei zu einem Problem? Wie lässt es sich beheben?

2. Passen Sie nun das Notenberechnungsprogramm so an, dass beim Start der Benutzer eingeben soll, wie viele einfach gewichtete Noten (z.B. Tests) erfasst werden sollen.

3. Um nun diese Noteneingabe durch den Benutzer zu realisieren benötigt man so genannte for-Schleifen und zur Speicherung der Eingaben Arrays. Betrachten Sie dazu das folgenden Beispielprogramm und erklären Sie den grundlegenden Aufbau einer for-Schleife, sowie die Deklaration und Initialisierung eines Arrays und wie auf Elemente in einem Arrays zugegriffen werden kann.

```
import java.util.Scanner;

public class AufgabenlisteManager {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Schritt 1: Array deklarieren und initialisieren
        // Wir wollen 3 Aufgaben (Strings) speichern.
        // Ein Array vom Typ String, das Platz für 3 Elemente bietet.
        int anzahlAufgaben = 3;
        String[] aufgabenliste = new String[anzahlAufgaben];

        System.out.println("--- Aufgabenliste erfassen ---");
        System.out.println("Bitte geben Sie " + anzahlAufgaben + " Aufgaben ein.");

        // Schritt 2: Das Array mit einer for-Schleife befüllen
        // Die Schleife läuft 'anzahlAufgaben'-mal.
        // 'aufgabenliste.length' gibt die Größe des Arrays zurück (hier 3).
        for (int i = 0; i < aufgabenliste.length; i++) {
            System.out.print("Aufgabe Nummer " + (i + 1) + ": ");
            // Liest die gesamte Zeile, die der Benutzer eingibt, als String.
            aufgabenliste[i] = scanner.nextLine();
        }

        System.out.println("\n--- Ihre erfassten Aufgaben ---");

        // Schritt 3: Den Inhalt des Arrays ausgeben, um das Ergebnis zu überprüfen
        // Eine weitere Schleife durchläuft alle Elemente des Arrays.
        for (int i = 0; i < aufgabenliste.length; i++) {
            System.out.println((i + 1) + ". " + aufgabenliste[i]);
        }

        scanner.close();
    }
}
```

4. Passen Sie nun abschließend so an, dass der Benutzer die Anzahl der einfach gewichteten Noten angibt und diese anschließend eingibt. Danach soll er das gleiche für doppelt gewichtete Leistungsnachweise eingeben.

5. Für Schnelle:
   (a) Ergänzen Sie Ihr Programm um die Ausgabe des entsprechenden Zeugnistext für die jeweilige Zeugnisnote:

Zeugnisnote	Zeugnistext
1	„sehr gut“
2	„gut“
3	„befriedigend“
4	„ausreichend“
5	„mangelhaft“
6	„ungenügend“

   (b) Passen Sie das Programm so an, dass der Benutzer anfangs keine Anzahl eingeben muss, sondern solange Noten eingeben kann, bis er beispielsweise ein ’x’ eingibt.

1.6 Für Experten: Genauere Betrachtung der Zahlendarstellung

1.6.1 Negative Zahlen

Um negative Zahlen binär darzustellen, gibt es die so genannte **Einerkomplement-** bzw. **Zweierkomplementdarstellung**.
Aufgaben:

1. Wie wird eine Binärzahl für das Einerkomplement umgewandelt? Welchen Wertebereich kann man mit 8 Bit mithilfe des Einerkomplement darstellen?

2. Nennen Sie zwei Nachteile, wenn für die Darstellung negativer Zahlen das Einerkomplement verwendet wird.

3. Wie wird eine Binärzahl für das Zweierkomplement umgewandelt?Welchen Wertebereich kann man mit 8 Bit mithilfe des Zweierkomplement darstellen?

4. Berechnen Sie mithilfe des Zweierkomplements schrittweise das binäre Ergebnis folgender Rechnungen. Wandeln Sie abschließend das binäre Ergebnis wieder in eine Dezimalzahl um. Verwenden Sie für binäre Darstellungen jeweils 8 Bit.
   (a) 20 - 5
   (b) 18 - 40

5. Gegeben sind die Binärzahlen (jeweils 6 Bit) u = 010011 und v = 001111 in der Zweierkomplementdarstellung. Berechnen Sie das Binär-Ergebnis von u + v. Hat bei der Addition ein Overflow (Überlauf) stattgefunden? Begründen Sie Ihre Antwort.

1.6.2 Reelle Zahlen

Für die binäre Darstellung von reellen Zahlen wird häufig die Norm **IEEE 754** verwendet.
Aufgaben:

1. Nach dieser Norm werden reelle Zahlen als Gleitkommazahlen und nicht als Festkommazahlen gespeichert. Nennen Sie einen Grund dafür.

2. Informieren Sie sich nun, wie reelle Zahlen durch IEEE 754 ins Binärsystem umgewandelt werden. Achten Sie auf folgende Begriffe: Exponent, Mantisse, einfache/doppelte Genauigkeit, Bias-Notation, normalisierte Form

3. Wandeln Sie folgende Zahl, die in 32-Bit Gleitkommadarstellung nach IEEE 754 gegeben ist, in ihre Dezimaldarstellung um.
   1 | 10000110 | 10100001000000000000000

4. Wandeln Sie die Dezimalzahl -42,125 in ihre Gleitkommadrastellung nach IEEE 754 mit einfacher Genauigkeit um.

5. Erklären Sie, warum bei folgendem Vergleich der Wert `false` auf der Konsole ausgegeben wird:

```
double a = 0.1 + 0.2;
System.out.println(a == 0.3);
```